KC87 Pico Recorder

von Thorsten Kattanek, letzte Änderung 24.10.2026

Inhaltsverzeichnis

Vorwort....2
Was ist der „KC87 Pico Recorder“ ?....2
Technisches „Bla Bla“....3
Wie sieht das Signal aus wenn es per „CSAVE“ aus dem KC87 kommt?....3
Wie können wir das Signal so umwandeln das wir es mit dem Pico empfangen und auswerten können?....4
Wie kommen wir an die Bits und Bytes?....4

Vorwort

Das Projekt entstand aus meinen Nachforschungen zum KC87. Ich habe mir 2025 einen KC87 zugelegt. Auch recht Preiswert, mit 100€ zu dieser Zeit. So oft findet man diese leider nicht mehr. Die Community ist natürlich deutlich kleiner als die der Commodore Liebhaber. Aber es gibt doch ein paar nette Internetseiten die ich hier mal Aufzähle.

https://hc-ddr.hucki.net/wiki/doku.php
https://www.sax.de/~zander/
https://floooh.github.io/virtualkc/

Das sind schon recht gute Quellen wenn man sich mit alten DDR Computern beschäftigen möchte. In dieser Dokumentation werde ich auch alle Technischen Details was die Elektronik und auch die Programmierung der Firmware und auch der Software für den PC wenn entspr. Tools von mir erstellt werden behandeln. Ich hoffe das am Ende ein nützliches, kleines Stück Hardware raus kommt womit man den KC87 erweitern kann. Am Ende würde ich dieses Projekt dann, mit in das ThePicoRetroTape (https://github.com/ThKattanek/the_pico_retro_tape) Projekt mit einfließen lassen. Aber das erst dann, wenn hier alles so funktioniert wie ich mir das vorstelle.

Was ist der „KC87 Pico Recorder“ ?

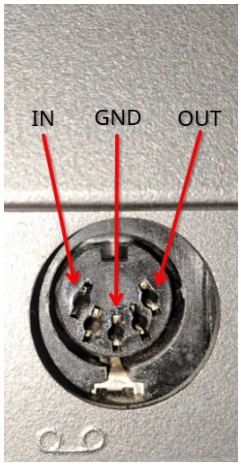
Mit dem Pico Recorder soll am Ende der „alte“ Kassettenrecorder ersetzt werden. Wie der Name schon erahnen lässt, basiert der „neue“ Recorder auf den Raspberry Pi Pico Borad.

Technisches „Bla Bla“

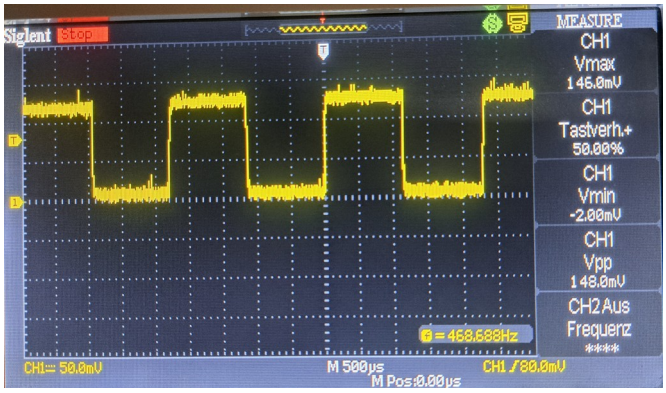
Wie sieht das Signal aus wenn es per „CSAVE“ aus dem KC87 kommt?

Das Kassetten Signal wird aus der rechten Seite des KC87 entnommen. Dort befindet sich eine Buchse wo man früher den Kassettenrecorder angeschlossen hat. Hier ein Bild wo man genau sieht wie diese Buchse standardmäßig belegt ist.



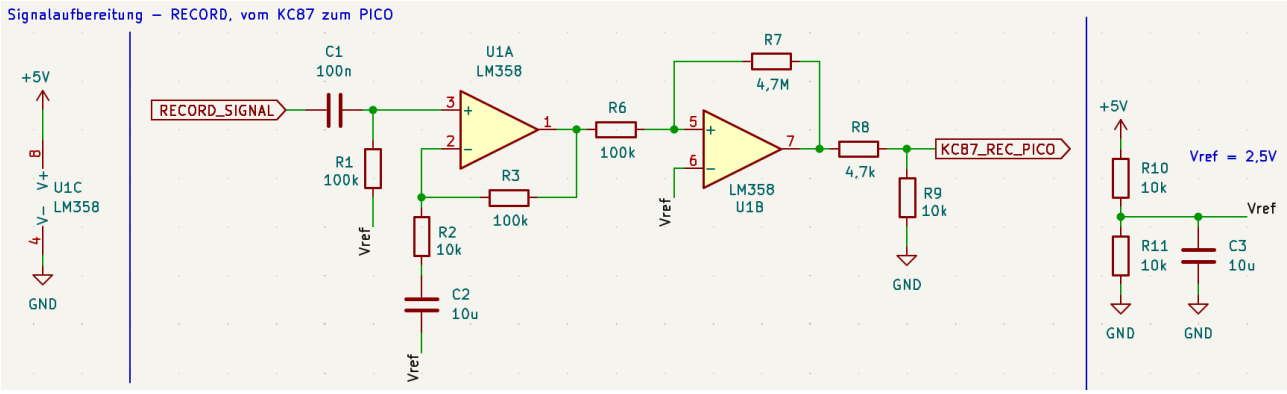


Das Signal was bei OUT raus kommt ist ein Rechtecksignal mit einer Amplitude von ca. 130-150mV. Laut Robotron ist das eine Frequenzmodulation bzw. Frequenzumtastung (FSK). Das ganze sieht mit dem Oszi so aus. Dieses Signal wird so ausgegeben wenn ich „CSAVE“ gleich nach dem Start eingeben und 2x mit Enter bestätige. Das ist nicht im BASIC ! Hier bleibt das Signal dauerhaft an.



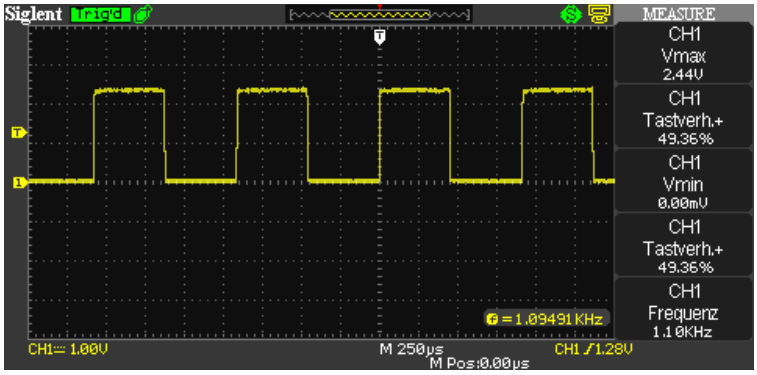
Wie können wir das Signal so umwandeln das wir es mit dem Pico empfangen und auswerten können?

Ich bin da sehr optimistisch und habe mich für folgende Schaltung entschieden. Das Signal vom KC87 ist sehr klein (ca. 130-150mV). Ich verstärke es deshalb zuerst mit einem Operationsverstärker und bringe es danach mit einem zweiten als Komparator auf einen Logik Level von 3,3V, um direkt an den Pico zu gehen. Beide stecken im LM358 (U1) und werden mit 5V VBUS versorgt.



Hier eine kurze Beschreibung der Schaltung:

Das Signal aus dem KC87 kommt über C1 (100n) auf den Eingang von U1A. Mit R1 und der Referenzspannung Vref = 2,5V (Spannungsteiler R10/R11, geglättet durch C3) liegt das Signal in der Mitte der Versorgungsspannung. U1A verstärkt es für das Wechselsignal um den Faktor 11 (1 + R3/R2). U1B arbeitet als Komparator gegen Vref. Die Hysterese über R6/R7 unterdrückt Rauschen. Der Ausgang wird durch den Spannungsteiler R8/R9 auf den Pegel des Pico angepasst. Ein Abgleich ist nicht nötig.
Hier das Ausgangssignal „KC87_REC_PICO“.



Gemessen am Pin „KC87_REC_PICO“ während des Vortons: Vmax = 2,44V, Vmin = 0V, Tastverhältnis ca. 49 %, Frequenz ca. 1,1 kHz.

Wie kommen wir an die Bits und Bytes?

Jetzt wo wir ein vernünftige Digitales Signal haben, können wir uns nun anschauen wie in diesem Datenstrom die Daten codiert sind.

Der Pico misst dazu mit seiner PIO-Einheit die Dauer jeder einzelnen High- und Low-Phase des Signals „KC87_REC_PICO“ am Eingang GPIO 2. Die Messwerte (Auflösung 0,2 µs) werden über USB an den PC geschickt und dort vom Programm „kc87tool“ ausgewertet. Die Aufnahme ist eine Tabelle mit den Pulsdauern, aus der sich Frequenzen und Bits ablesen lassen.

In einer echten „CSAVE“-Aufnahme findet man genau drei verschiedene Pulslängen. Immer zwei gleich lange Pulse (High und Low) bilden eine Periode:

Bit 0: 2 Pulse mit je ca. 250 µs, das sind ca. 2 kHz.

Bit 1: 2 Pulse mit je ca. 455 µs, das sind ca. 1,1 kHz.

Trennzeichen: 2 Pulse mit je ca. 870 µs, das sind ca. 575 Hz.

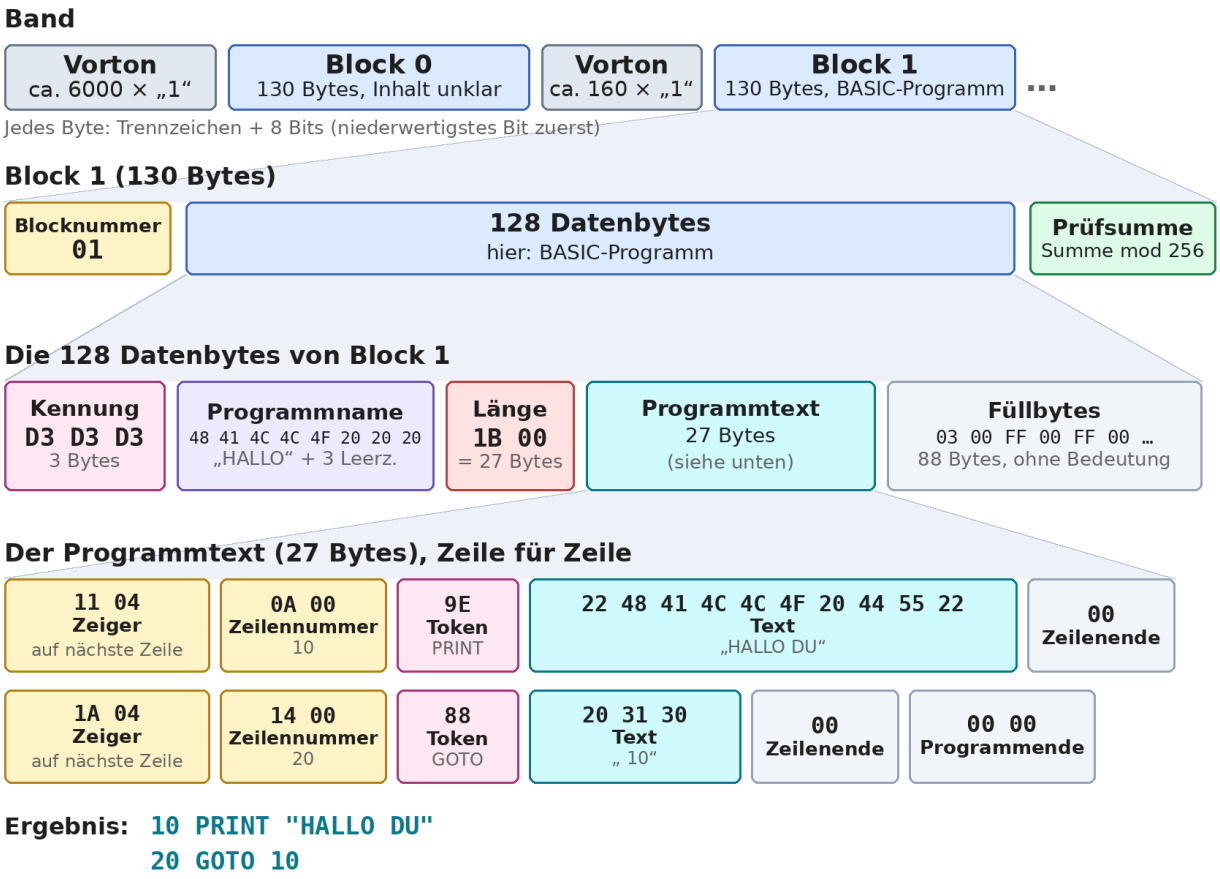
Ein Byte besteht aus 8 Bits, das niederwertigste Bit kommt zuerst. Vor jedem Byte steht ein Trennzeichen. Am Anfang jedes Blocks steht ein Vorton aus lauter 1-Bits, vor dem ersten Block ist er mit ca. 6000 Perioden besonders lang.

Die Daten sind in Blöcken zu 130 Bytes organisiert: Zuerst kommt die Blocknummer, danach 128 Datenbytes und zum Schluss eine Prüfsumme. Die Prüfsumme ist die Summe der 128 Datenbytes modulo 256. Bei meiner Aufnahme stimmten die Prüfsummen beider Blöcke.

Ein mit „CSAVE“ gespeichertes BASIC-Programm beginnt mit den drei Bytes D3 D3 D3. Danach folgt der Programmname mit 8 Zeichen und dann der BASIC-Programmtext. Bei einem Test mit dem Namen „HALLO“ stand dort 48 41 4C 4C 4F 20 20 20, also „HALLO“ mit drei Leerzeichen am Ende.

Das Programm „kc87tool“ macht diese Auswertung mit dem Befehl „decode“: Es zeigt für jeden Block die Nummer und ob die Prüfsumme stimmt, bei BASIC-Programmen auch den Namen. Mit der Option „--tap“ schreibt es die Blöcke mit gültiger Prüfsumme in eine TAP-Datei. Diese TAP-Datei habe ich im Emulator JKCEMU (Version 0.9.9.1) geladen und das Programm lief. Getestet habe ich bisher nur ein BASIC-Programm. Maschinencode-Dateien habe ich noch nicht ausprobiert.

Hier noch einmal der Aufbau der Datei, die ich beim „CSAVE“ empfange. Als Beispiel dient ein Programm mit dem Namen „HALLO“ und den beiden Zeilen 10 PRINT "HALLO DU" und 20 GOTO 10:



Ergebnis:
```
10 PRINT "HALLO DU"
20 GOTO 10
```

Block 0 enthält in meinem Beispiel fast nur Füllbytes und ein paar Reste. Was der KC87 damit vorhat, weiß ich noch nicht. Das Programm steckt in Block 1.

Die Länge (1B 00 = 27) zählt den Programmtext mit den beiden Endebytes 00 00. Der Zeiger am Anfang jeder Zeile zeigt auf die nächste Zeile, es ist eine Adresse im Speicher des KC87. Befehle wie PRINT (9E) und GOTO (88) sind als einzelnes Byte, als sogenanntes Token, gespeichert. Alles nach dem Programmende bis zum Blockende sind Füllbytes. Wie ein Programm aussieht, das länger als 128 Bytes ist, habe ich noch nicht getestet.